Git tutorial

Git is a widely popular tool for version control. This tutorial teaches the core concepts of Git.

Table of Contents

Part 0: Setting up Git and a Git repository

- Installing Git
- Configuring Git
- Adding an SSH key to your Bitbucket account
- Setting up a Git repository

Part 1: Introduction to Git

- What is Git?
- Overview of making changes to the remote repository
- Basic Git workflow diagram
- The four stages for tracking changes
- Commands for moving changes
- Basic workflow for moving changes to the remote repository

Part 2: How to Use Basic Git Commands

- What are commits
- Checking the status of a project
- Adding files to the staging area
- Creating a new commit
- ava log
- Additional information about commits
- Practice making a commit
- Pushing commits to a remote repository
- Retrieving commits from a remote repository
- Resolving merge conflicts when they occur
- Restoring old versions of files

Part 3: How to Use Intermediate Git Commands

- Branching tutorial using website
- Branches
- Creating branches
- Git switch
- Branches and commiting
- Merge commits
- Additional Information

Part 0: Setting up Git and a Git repository

To learn how to set up Git and a Git repository, go to `reference-materials/0-setting-up-git.md`

Part 1: Introduction to Git

What is Git?

Git is a distributed version control system (VCS) that helps developers track changes in code, collaborate with others, and manage different versions of a project efficiently.

In Git, all files and all their past versions are stored in a main repository referred to as the `remote repository`. Each developer has a copy of this repository on their local machine. This repository is referred to as the `local repository`. You can make as many changes to the local repository as you want. Then, when you are connected to the internet, you can push these changes to the remote repository in order to update it.

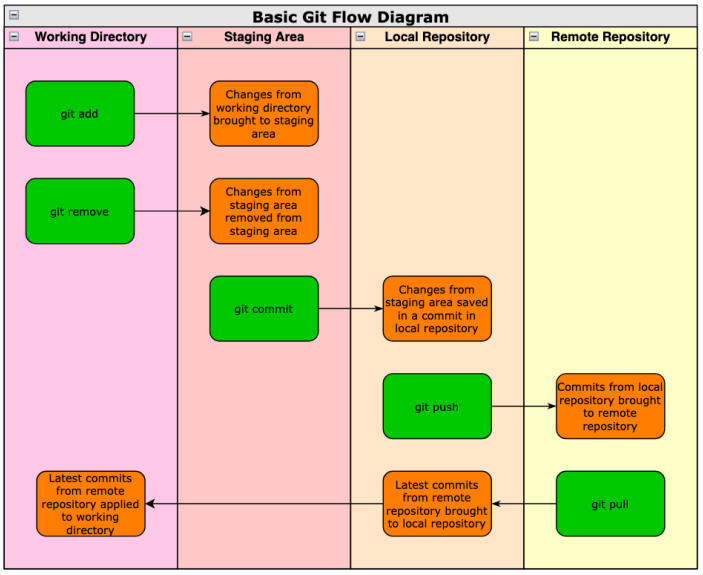
Overview of making changes to the remote repository

Here's a general overview of making changes to the remote repository:

1. Create new files or make changes to the files currently available to you.
    - The files currently available to you are referred to as the `working directory`.
2. Add these changes to the staging area.
    - The `staging area` is where these changes are put when you are ready to commit them.
3. Create a commit using the changes from the staging area.
    - A `commit` is basically a "save" or a "snapshot" of the code.
    - When a commit is created, it is added as the latest commit of the local repository. (This basically means that the commit is the latest "version" of the repository.)
4. Push the commit to the remote repository from your local one.
    - When a commit is added to the remote repository, it also becomes the remote repository's latest commit.

Basic Git workflow diagram

Here's a basic diagram of the previous section. It can be helpful to consult as you read the next few sections. (This diagram also includes pulling from the remote repository, which updates both the local repository and the working directory with the changes from the remote repository.)



The four stages for tracking changes

In Git, there are four stages for tracking changes to be aware of:

1. The Working Directory
    - This refers to the set of files that are currently available to you.
    - It includes both changes that you have put in the staging area and changes that you have not.
2. The Staging Area
    - When you modify or create new files, you can add them here
    - When files are in the staging area, they can be modified in the working directory, but those additional changes will not be added to the staging area unless you intentionally add them.
3. The Local Repository
    - This contains all past commits for your project.
    - Changes from the staging area can be added to your local repository by creating a commit with them. (Commits are basically "saves" or "snapshots" of your repository.)
4. The Remote Repository
    - This contains all past commits that have been added the remote repository.
    - Commits are only added here when someone adds them from their local repository.

Commands for moving changes

In Git, there are a series of different commands that allow the developer to move changes between the working directory, the staging area, the local repository, and the remote repository.

The main commands are as follows:

- `git status`
    - Get which changes are and are not in the staging area.
- `git add <file path>`
    - Add changes to the staging area.

- `git rm <file path>`
    - Remove changes from the staging area.
- `git commit`
    - Create a commit with the changes from the staging area.
- `git push`
    - Push the commits from the local repository to the remote repository.
- `git pull`
    - Pull the commits from the remote repository and add them to your local repository.
        - This may involve performing a merge if your files conflict with the remote ones.

Basic workflow for moving changes to the remote repository

As a result, the general workflow for moving changes to the remote repository is as follows:

1. (Optional) Use `git pull` to make your local repository and working directory up to date with the remote repository.
2. Create new files or make changes to existing ones.
3. Use `git` status to check what files have been created or modified.
4. Use `git add` to add the changes you choose to the staging area.
5. Use `git status` to verify that you have added the changes you wanted to the staging area.
6. Use `git commit` to create a commit from the changes in the staging area.
    - This commit becomes the latest commit in your local repository.
7. Use `git status` to make sure all of the changes were committed.
8. Use `git push` to push the commits from your local repository to the remote one.
9. (Optional) Use `git pull` to pull from the remote repository to make sure your local repository is still up to date.

Bear in mind that your repository does not have to be up to date with the remote repository to make changes. That said, if your local repository is too different from the remote repository, you may have to resolve conflicts between it and your local repository in order to push these changes.

Additionally, you do not have to add all of the changes from your working directory to the staging area if you do not want to add all of the changes to a commit.

Part 2: How to Use Basic Git Commands

What are commits

In Git, `commits` refer to different versions of a project. Rather than saving one file at a time and having a different version for each file, Git allows you to save the entire state of the project at once in a commit.

Once a commit is created, it cannot be modified.

This means that, when you create or edit new files, those changes do not automatically apply to a new commit. Instead, you have to put those changes in the staging area and then create a new commit with the

changes in order for them to be saved.

Checking the status of your project

The command `git status` tells you what files from the working directory you have created or modified and whether those changes are in the staging area or not.

Check the current status of your project:

```
git status
```

You get the message:

```
On branch main
nothing to commit, working tree clean
```

This tells you that you don't have any files modified or created in your working directory.

You use `git status` any time you want to know what files you have modified or created and whether they are in the staging area.

Create a new file:

```
echo "example" > A
git status
```

You get the message:

```
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        A
```

This tells you that you have created or modified the file 'A', but it is not in the staging area.

Adding files to the staging area

To add this file to the staging area, use the command `git add`.

```
git add A
git status
```

Now, you get the message:

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   A
```

This means that the file A is in the staging area.

Creating a new commit

To create a new commit, use the command `git commit`:

```
git commit -m "Added file A."
git status
```

Now, you get the message:

```
On branch main
nothing to commit, working tree clean
```

This means that you have made a new commit and can continue creating more changes.

You can also just do:

```
git commit
```

This allows you to write a longer message than if you had included the `-m` argument.

ava log

Briefly, I would like to introduce the command `ava log`. This is not a Git command. Instead, it is available through the OSW command-line tool Ava, which comes by default on all OSW machines.

`ava log` shows your history of commits in a tree.

`ava log <file path>` show your history of commits for a specific file in a tree.

Run:

```
ava log
```

You should see something like this:

```
* 6adb560 (HEAD -> main) Added file A.
* dc542db (origin/main, origin/HEAD) Added file example.txt.
```

This shows you the history of the commits that exist in the repository. The first one `Added file example.txt` is a commit that I originally added to the remote repository. The second one `Added file A.` is a commit that you added to your local repository.

Additional information about commits

For some additional information, commits, besides storing a "save/snapshot" of a repository, contain helpful metadata such as:

- Author
- Timestamp
- Commit message
- Commit hash

When we ran `ava log` above, we got the following:

```
* 6adb560 (HEAD -> main) Added file A.
* dc542db (origin/main, origin/HEAD) Added file example.txt.
```

This shows us two important pieces of information - the `commit hash` and the `commit message`.

The commit hash is a unique 40-character identifier that exists for each commit. When a commit hash is only its first 7 characters, we refer to it as a `short hash`. If we ever want to switch back to a commit, we can use its commit hash, short hash, or revision number (which is a concept that is addressed in the Ava tutorial.) Two examples of short hashes above are `6adb560` and `dc542db`.

You don't have to keep these commit hashes written down as you can use `ava log` to see the history of commits. That said, over time, the history can become very long. If there is a commit that is particularly important to remember, it can be helpful to put its hash in a file to keep track of it.

The commit message is a message created by the author when making a commit. Two examples of commit messages above are `Added file A.` and `Added file example.txt.`

Practice making a commit

For practice, make another commit:

This time, you're going to make use of the `git remove` command, which removes changes to a file from the staging area.

```
echo "example" > B
echo "random" > random
git status
git add B
```

```
git status
git add random
git status
git remove random
git status
git commit -m "Added file B."
git status
ava log
```

As you can see, there are now three commits in your git history.

If you ever want to see which files have been created or modified and whether they are in the staging area, just remember to use `git status`.

Pushing commits to the remote repository

Your commits are currently only on your local repository.

The remote repository contains my commit `Added file example.txt.`, but it does not contain your commits `Added file A.` and `Added file B.`.

To push these commits to the remote repository, run the command:

```
git push
```

This command adds your commits to the remote repository. Now, the latest commit on the remote repository is your most recent commit.

Retrieving commits from the remote repository

If someone else adds a commit to the remote repository, it is not automatically brought onto your local repository.

On my machine, let me add a new commit to the remote repository:

```
echo "example" > C
git status
git add C
git status
git commit -m "Added file C."
git status
ava log
git push
```

I just pushed the commit to the remote repository. For you to retrieve the commit, you have to run this command:

```
git pull
```

Now, you just applied my commit to your repository.

`git pull` takes the commits from the remote repository and applies these commits to your local repository.

Resolving merge conflicts when they occur

Sometimes, you might edit a file and someone else might also edit the same file. If they push their commits to the remote repository, you might be prevented from also pushing your commits to the remote repository.

To resolve this issue, you will have to pull these commits. Sometimes, when you do so, you will get a merge conflict. To fix this, you will have to use an IDE or a text editor to manually resolve these merge conflicts. (The merge conflict is visible in the file and shows their version and your version. You have to choose which changes you want.)

This should generally be a rare occurrence in mission operations.

We'll do an example of this so I can show you.

On your machine:

```
echo "I like the color red." > opinion
git status
git add opinion
git status
git commit -m "Created file opinion."
git push
```

On my machine:

```
echo "I like the color yellow." > opinion
git status
git add opinion
git status
git commit -m "Modified the file opinion."
git push
```

On your machine

```
echo "I like cars a lot." > opinion
git status
git add opinion
git status
```

```
git commit -m "Updated the file opinion."
git push
```

Your push will fail.

Here's what you need to do:

```
git fetch
git merge
```

Now, you will need to resolve the merge conflict by opening the file in an IDE or text editor of choice. Simply, choose over-write the merge conflict with the changes you want in the file.

Now, you can push your commit.

```
git status
git add test
git status
git commit -m "Resolved merge conflict. Updated the file opinion."
git push
ava log
```

Restoring old versions of a file

Sometimes, you might want to replace a file with a previous version of itself. To do this, you simply need the commit hash or revision number of that file. To retrieve the file's commit hash let's do this:

```
ava log opinion
```

Select a lower commit hash (which is the older one). Then, run this command:

```
git restore --source <commit hash or revision number> -- opinion
```

Now, you have the older version of the file. You can add this file to your next commit if you want.

If you want to undo this change, simply run the command.

```
git restore -- opinion
```

This will restore the file opinion to how it was in the previous commit.

Part 3: How to Use Intermediate Git Commands

Branching tutorial using website

First, start by using this website: https://learngitbranching.js.org

Do specifically:

- Main Introduction Sequence 1, 2, 3
- Main Ramping Up 1, 2, 3
- Remote Push & Pull -- Git Remotes! 1, 2, 3, 4, 5, 6

If you get stuck on anything, don't sweat it. I can walk you through it.

Summary of what you just learned:

- Saves in git are stored as commits.
- Commits can branch off of each other.
- Branches can merge back into other branches.
- Branches and `HEAD` are really just pointers to commits.
- HEAD always points to what commit you are on. (This is the commit your working directory is based on.)

Branches

Branches tracks the latest commit made on that branch. The default branch in a new repository, which you are on, is called `main`. Because you are on main, every time you add a commit, the branch pointer main moves to the current commit. HEAD also moves to the current commit because HEAD always points to what commit you are on.

Creating branches

Create a new branch:

```
git branch
git branch newFeature
git switch newFeature
git branch
```

`git branch` shows you which branch you are on and what branches exist.

`git branch <branchName>` create a new branch from HEAD. It does not automatically switch you that branch.

`git switch <branchName>` switches you to that branch.

From `git branch`, you can see that you are now on the newFeature branch.

Run:

```
ava log
```

The top line of your output looks something like:

```
* 164fa2e (HEAD -> newFeature, main) Added file C.
```

This means that your commit has the pointers newFeature and main pointed at it. You are currently on the newFeature branch because HEAD is pointing at it. If you create a new commit, main will stay pointing at this commit, while HEAD and newFeature will point at the new commit.

Git switch

`git switch` moves you to a different branch.

```
git branch
git switch main
git status
git branch
ava log
```

As you can see from `git branch` and `ava log`, you are now on branch main and HEAD points to main.

Branches and committing

As you make commits while on the branch newFeature, the branch will follow along with each new commit:

```
git switch newFeature
echo "D" > D
git add D
git commit -m "Added file D."
ava log
```

Notice how the `main` branch is still pointing at the previous commit, but `newFeature` is pointed at commit `Added file B.`. As you make commits while on the branch, the branch will follow along with your new commits.

Let's go back to main and make a commit there.

```
git switch main
echo "E" > E
git add E
git commit -m "Added file E."
ava log
```

Notice how the `main` branch is now pointing at the commit `Added file E.`, while `newFeature` is pointed at the commit `Added file D`. The two branches have diverged. Both `Added file D.` and `Added file E.` share the previous (parent) commit `Added file C.` Let's merge them back together.

Merge commits

Merge commits are commits that have more than one parent. Merge your two commit branches by inserting a commit which uses both of them as parents.

First, run `ava log`:

```
`ava log`
```

This command will tell you that you are on the main branch. Copy the commit hash from the latest commit on the newFeature branch.

Then, use that commit hash for the merge:

```
git merge <commit hash>
git commit -m "Performed merge."
ava log
```

As can be seen from `ava log`, we now have the commit `Performed merge.` whose parents are the commits `Added file D.` and `Added file E.`, and their parent is the commit `Added file C.`.